

Performance Analysis of Federated Learning

Full Fine-Tuning vs. LoRA Fine-Tuning in Complex Networks

Student: Marakis Michail

Professor: Dr. Koutsopoulos Iordanis

Athens University of Economics and Business (AUEB) | Department of Informatics

Course: Performance Analysis of Complex Networked Systems (8th Semester)

Presentation Overview

01

Theory & Architecture

Federated Learning Foundations, operational cycles, advantages and architectural drawbacks.

02

Fine-Tuning Strategies

Analysis of Full Fine-Tuning (FFT) Vs parameters-efficient Low-Rank Adaptation (LoRA).

03

Experiments

Analyzing the conditions for both the local and the distributed experiment.

04

Results & System Performance

Results and observations from experiments.

05

Simulation & Future

Simulation of the FL Fine-Tune network as an M/G/1 queue. Little's Law and Quantum Federated Learning (QFL)

01

PART 1

Federated Learning Foundations

What is Federated Learning?

Definition: Federated Learning (FL) is a distributed machine learning approach where the model is trained locally on edge devices.

▪ Data Privacy

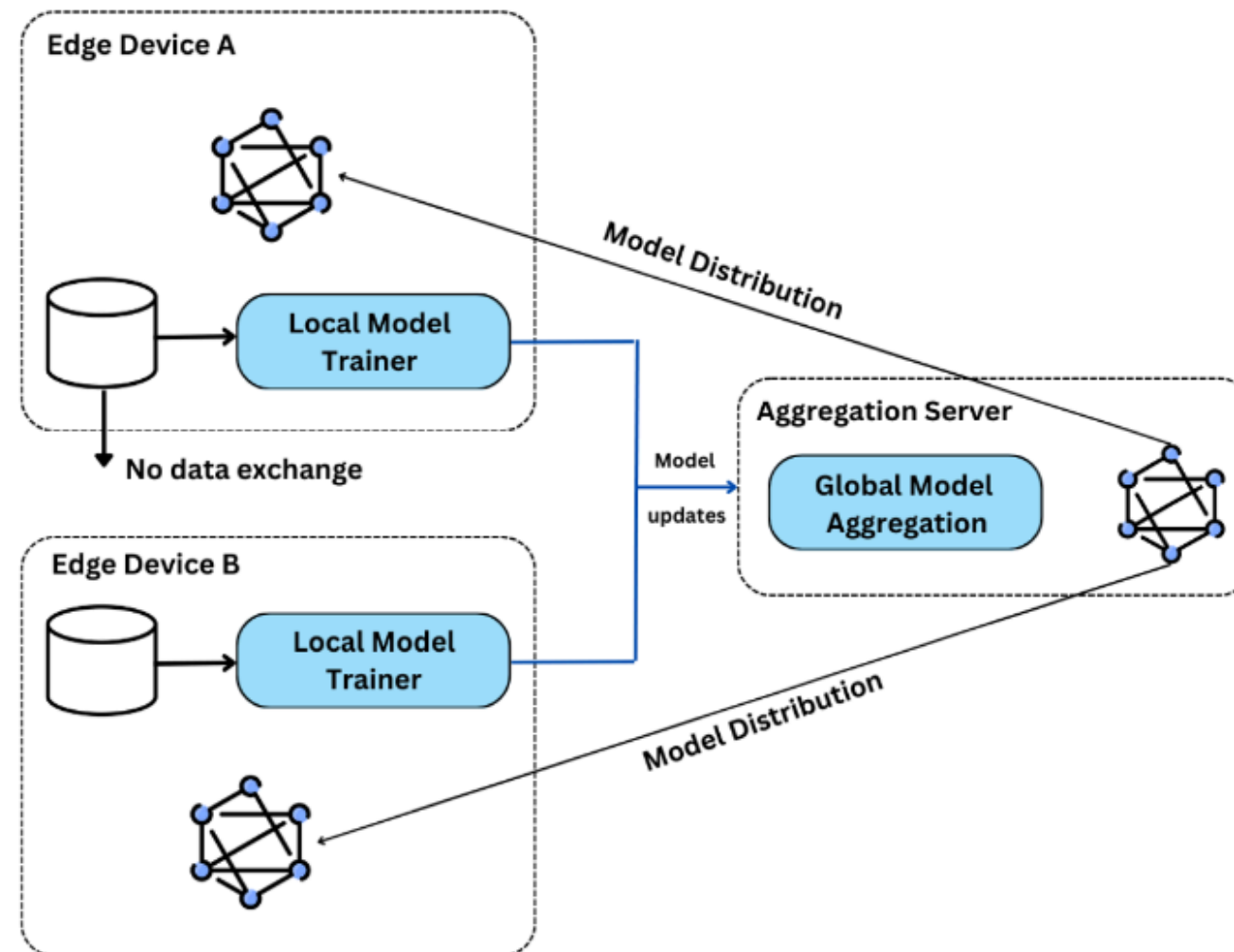
Raw user data stays strictly on the local device. This ensures complete data isolation and thus privacy for the local dataset.

▪ Decentralization

Removes the need for expensive cloud storage or central databases. By keeping data distributed across devices, we remove major security risks as well.

▪ Independent Training

Edge clients train independently on their own local data without communicating with each other. They only upload the weights from the global model that each has trained to the central server.



The FL Operational Cycle

01

Global Broadcast

The Server selects a specific subset of active clients and broadcasts the current global model weights to them.

02

On-Device Training

Each client starts the training by processing gradients locally. The raw data from the local dataset are never leaving the device and are not accessible to anyone apart from the client.

03

Weight Aggregation

After finishing training each client transmits the weight updates back to the server. The server uses algorithms like FedAvg to combine each trained group of weights with the global model.

Advantages of FL



Privacy

Federated Learning keeps the user data on local edge devices as also said before. As a result it matches privacy laws like GDPR. Since training happens entirely on the device locally and files never leave the local device and the network, so the risk of data leaks is very minimized.



Bandwidth Efficiency

This architecture prevents the network from getting blocked by very heavy files. Instead of uploading gigabytes of raw training data from edge clients to a central server, devices only transmit relative small in size weight updates. With this way the network is more protected from overloads.



Real-Time Answers

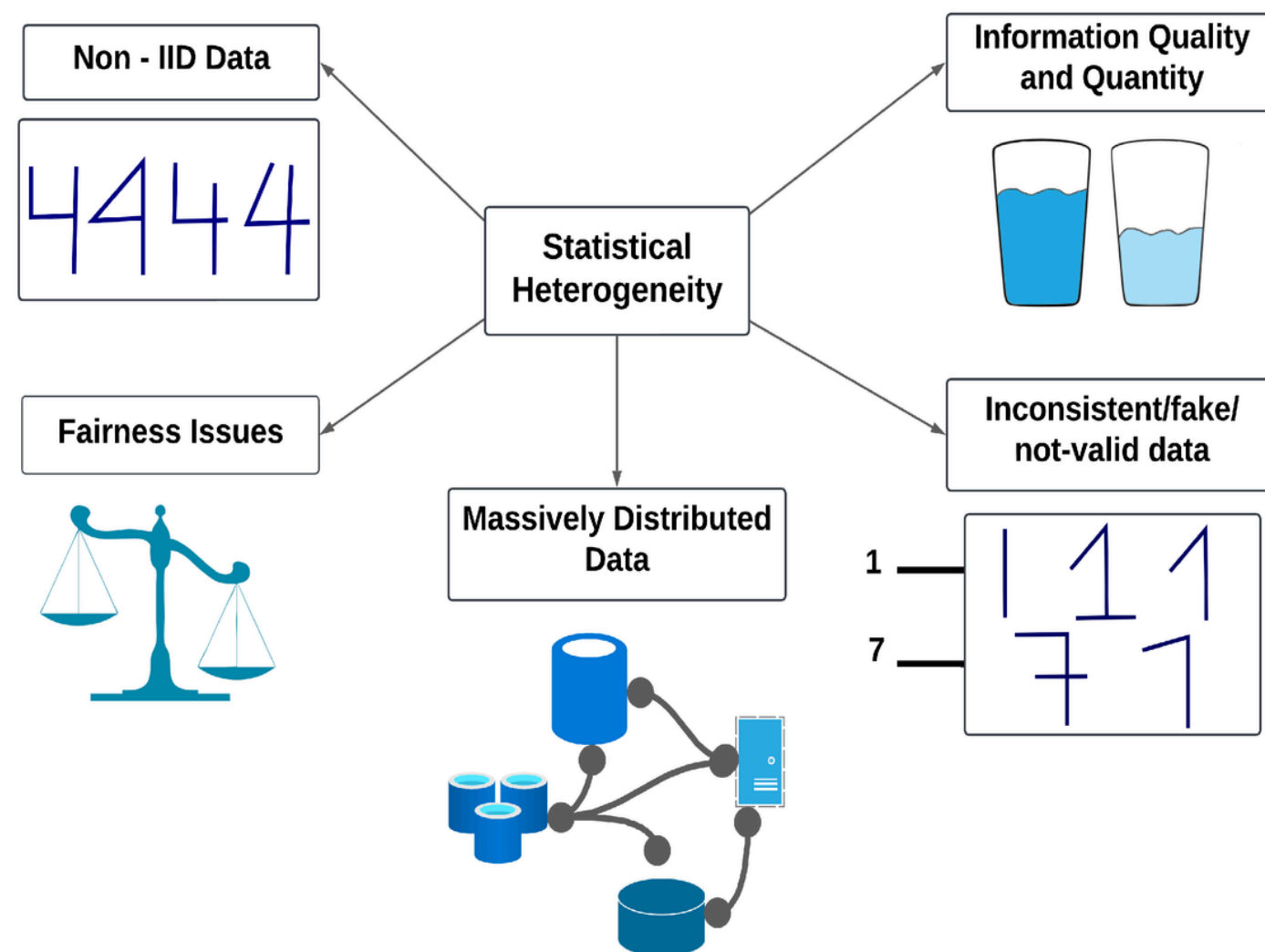
The final trained AI model runs completely inside the user's own device. This allows the system to give machine learning answers instantly with zero network lag.

Architectural Challenges in FL

Statistical Heterogeneity

Every device has different dataset based that is based on the environment of each client, making the local datasets completely different from each other.

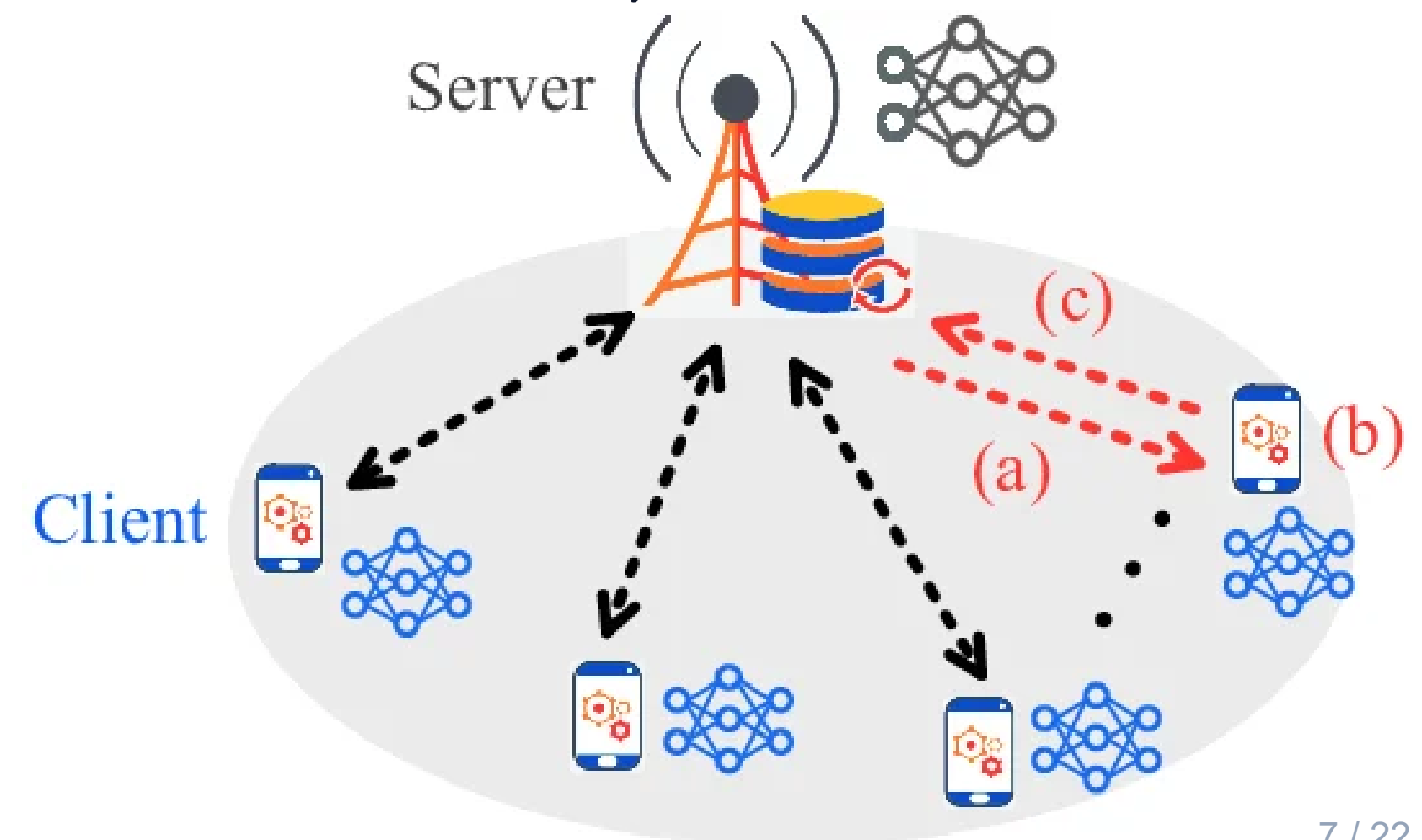
Because of this data mismatch, local AI models learn different things. This may confuse the central server during weight aggregation. As a result it could cause severe synchronization delays or even make the global model fail to converge.



Network Volatility & Stragglers

Edge device operate over different consumer networks and thus some of them could face more unstable upload speeds and sudden packet drops than others.

These devices that experience heavy connection drops or slow hardware performance are called the stragglers of the system. These slow nodes, hold back the central server's synchronization loop, creating latency bottlenecks across the entire system.



02

PART 2

Fine-Tuning Strategies

Full Fine-Tuning (FFT)

Unlimited Parameter Modification

FFT updates the entire model during training by adjusting all the weights in every layer. This means that every part of the neural network is modified step by step during backpropagation.

Huge Computational & Memory Overhead

Managing this kind of updates requires strong CUDA-enabled hardware. GPUs need a lot of memory (VRAM) to store things like model weights, gradients, and intermediate data (for example feature maps for CNNs, hidden states for RNNs...) during training.

Checkpoint size

A checkpoint is a saved snapshot of the model during training that contains its learned parameters (weights). Checkpoints are used to resume training or share trained models. The size is calculated as:

Checkpoint Size = $P \times B$ where:

- **P** = total number of trainable parameters
- **B** = number of Bytes required per parameter, depending on the numerical precision used. For example in FP32 which is a typical format for the parameters, each parameter costs 4 Bytes.

Network Overload

Uploading large model files from edge devices over normal internet connections, it could create major network instabilities and straggles during the uploading time of weights from clients to server. This leads to high latency and makes large scale implementations way more difficult.

Low-Rank Adaptation (LoRA)

Mathematical Foundation

LoRA systematically freezes base structural neural layers, tracking optimization updates by decomposing the initial weight Matrix into two low-rank parameter matrices:

Where:

- $W_0 \in \mathbb{R}^{(d \times k)}$ (Frozen foundational weight tensor)
- $B \in \mathbb{R}^{(d \times r)}$ and $A \in \mathbb{R}^{(r \times k)}$ (Trainable Low-Rank Matrices)
- $\text{rank } r \ll \min(d, k)$ restricts weight rank updates.

$$W = W_0 + \Delta W = W_0 + (B \times A)$$

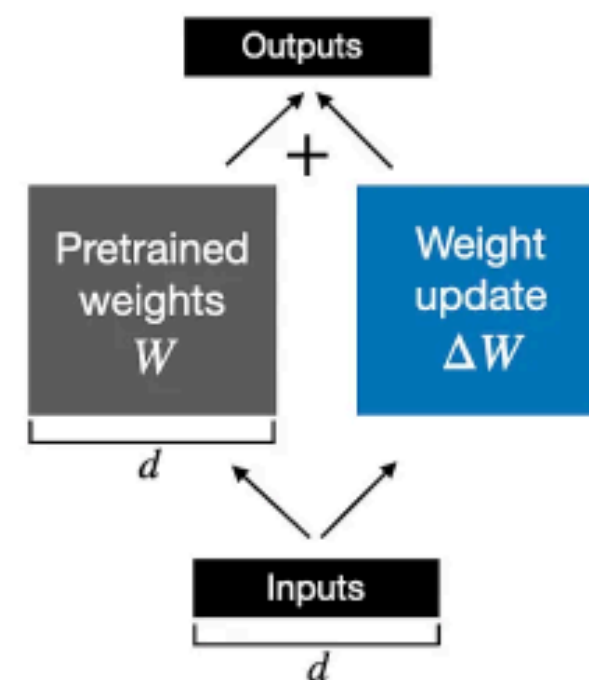
$$\begin{bmatrix} w1 & w4 & w7 \\ w2 & w5 & w8 \\ w3 & w6 & w9 \end{bmatrix} = \begin{bmatrix} b1 \\ b2 \\ b3 \end{bmatrix} \times \begin{bmatrix} a1 & a2 & a3 \end{bmatrix}$$

Original Matrix W Matrix B Matrix A

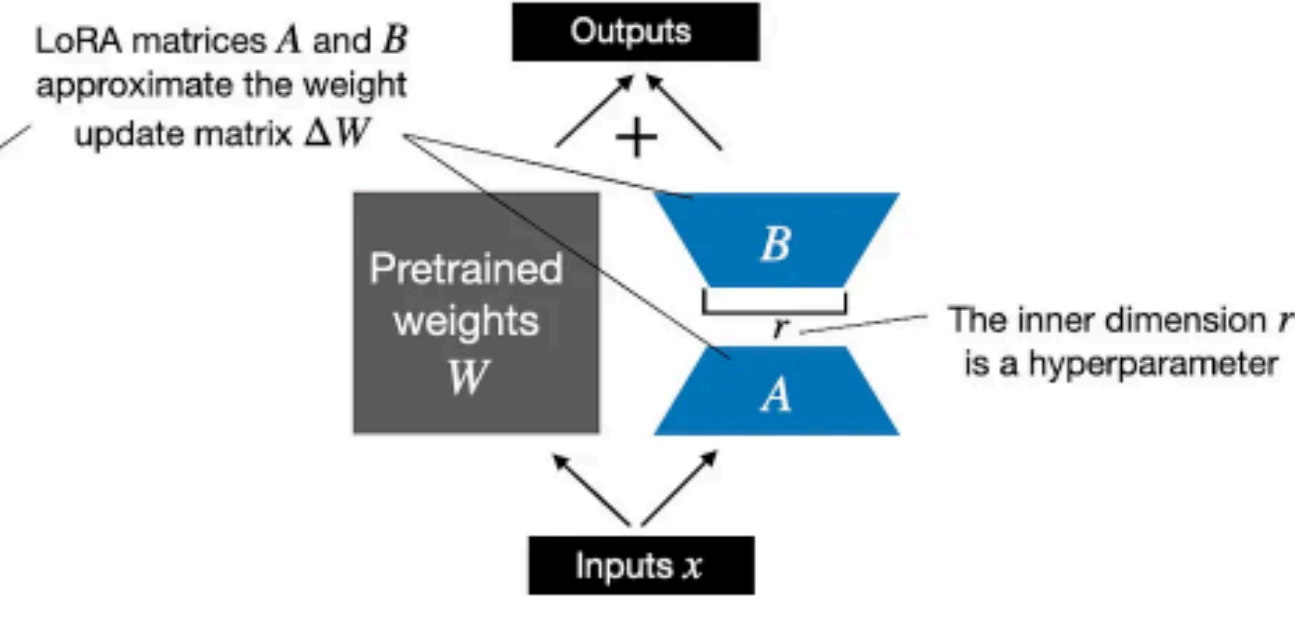
Results of the Architecture

LoRA reduces the number of trainable parameters by only updating small matrices (A and B), cutting them up to 99.75%. The percentage depends on the chosen rank (r), which defines the low-dimensional size of the A and B matrices used to approximate weight updates. This makes training much more efficient. More importantly, this efficiency comes with only minimal loss in accuracy often around 3%-6%.

Weight update in **regular finetuning**



Weight update in **LoRA**



Difference in a Hypothetical Scenario

Suppose we want to Fine Tune **Llama-3.1 8B**

Tuning Method	Trainable Parameters	Checkpoint Size	Network Channel Load
Full Fine-Tuning	8.000.000.000 (100% of the Model)	16 GB (Massive Storage, Whole model)	Very Heavy for the Network
LoRA Fine-Tuning	4.000.000 - 20.000.000 (0.05 - 0.25% of the Model)	15 - 80 MB (High Compression, Low Rank Matrices)	Easier to work with

Summary: As we can see from the table, LoRA reduces the amount of data that needs to be transferred and processed during the training loop. By minimizing trainable parameters and checkpoint size, it allows faster training times and also weaker devices as well to participate more in fine-tuning systems.

03

PART 3

Experiments

Experiment 1: Localized Setup

Goal

- **Performance:** Running the server and the clients on the same computer eliminates network delays and thus it is expected to finish much faster than the distributed training.
- **Stress-Testing Hardware:** The main goal was to find the heaviest possible model, dataset, and hyperparameters that the hardware could handle without triggering the Out-Of-Memory (OOM) error.

Hardware & Frameworks

- **GPU:** NVIDIA GeForce RTX 4060 (8GB VRAM).
- **Main Frameworks:** Flower, PyTorch, peft

Model & Dataset

- **Model:** Tiny-BERT (14.5 million trainable parameters).
- **Dataset:** rotten_tomatoes (approx. 480.000 movie reviews labeled “fresh” or “rotten”).)

Hyperparameters

- **Rounds:** 10 global federation rounds.
- **Clients:** 10 simulated client partitions with 100% participation.
- **Training:** 10 local epochs per round with fixed batch_size = 16
- **Dynamic Learning Rate:** Starts with 1e-5, then drops to 5e-6 after Round 2 for more stable convergence.
- **Allocated Resources:** 2 CPUs and 1.0 GPU per simulated client partition forcing sequential training of the 10 clients to prevent VRAM Out-Of-Memory (OOM).
- **LoRArank:** $r = 16$. Using this r , the results are more comprehensive.

Experiment 2: Distributed Setup

Goal

- **Real-World Testing:** By distributing the server and the clients across 2 different computers, it is possible to measure performance over a Wide-Area Network. Therefore, adding real-world internet noise and transport delays into the FL system the experiment becomes even more realistic and interesting.
- **Network Impact:** Moving data outside a single machine introduces network latency and creates unstable connections. As a result it is expected to have slower training times than the local experiment.

Hardware & Frameworks

- **Server:** NVIDIA GeForce GTX 1650 (4GB VRAM). Located in Markopoulo, Athens
- **Clients:** NVIDIA GeForce RTX 4060 (8GB VRAM). Located in Zografou, Athens
- **Main Frameworks:** Same code as the local. Using **ngrok API** to make it distributed.

Hyperparameters & Model & Dataset

- **Fair Comparison:** In order to have as fair as possible comparison, the Model (Tiny-BERT), Dataset (rotten_tomatoes), and some hyperparameters (Rounds=10, Dynamic LR and $r = 16$ for LoRA) remain exactly the same with the localized setup. Due to weak hardware on the server and many problems with the network, the results are being presented with clients = 2 and Batch Size = 8.

Network Layer

- **Secure Tunneling:** The server and clients are connected through the **ngrok API**, which provides a secure communication tunnel and allows them to work together across firewalls and local NAT networks.

04

PART 4

Results & System Performance

Localized Results

Round	Throughput in MB (FFT)	Throughput in MB (LoRA)	Accuracy (FFT)	Accuracy (LoRA)
1	16.73	0.06	0.52	0.51
2	33.46	0.13	0.58	0.52
3	50.2	0.19	0.59	0.51
4	66.93	0.25	0.59	0.52
5	83.66	0.32	0.58	0.53
6	100.39	0.38	0.57	0.54
7	117.12	0.44	0.58	0.55
8	133.86	0.51	0.58	0.54
9	150.59	0.57	0.58	0.55
10	167.32	0.63	0.57	0.55

FFT Total Training Time: 4.37mins

LoRA Total Training Time: 3.12 mins

Observation: Looking at the results it is observable that by Epoch 10, LoRA reduces the throughput by 99.62% (0.63 MB vs 167.32 MB) compared to Full Fine-Tuning. On the other hand, this bandwidth saving comes with a small accuracy drop of just 3.5% (0.55 vs 0.57). Another major difference is the training time where LoRA takes approx. 29% less time to finish. These results are expected as we saw at the theoretical comparison. Also, this proves LoRA can indeed eliminate bandwidth problems while keeping the accuracy relative high and close to FFT's.

Distributed Results

Round	Throughput in MB (FFT)	Throughput in MB (LoRA)	Accuracy (FFT)	Accuracy (LoRA)
1	34.50	0.13	0.55	0.53
2	68.22	0.26	0.59	0.55
3	102.10	0.39	0.61	0.56
4	136.45	0.52	0.61	0.57
5	170.80	0.65	0.60	0.58
6	204.38	0.77	0.59	0.58
7	238.90	0.90	0.60	0.59
8	273.10	1.03	0.60	0.59
9	307.52	1.16	0.60	0.59
10	340.14	1.42	0.63	0.59

FFT Total Training Time: 14.46 mins

LoRA Total Training Time: 10.58 mins

Observation: Looking at the results it is observable that by Epoch 10, LoRA reduces the throughput by 99.58% (1.42 MB vs 340.14 MB) compared to Full Fine-Tuning. On the other hand, this bandwidth saving comes with a small accuracy drop of just 6.3% (0.59 vs 0.63). Another major difference is the training time where LoRA takes approx. 26.8% less time to finish. These results are expected taking consideration the fact that is in distributed setup with only 2 clients. This increase in time of training and throughput, show the difficulties of real life applications compared to localized simulations. Even with 8 clients less, both throughput and training time are way higher due to the the difficulties and the constraints from the network communications. An interesting observation is the accuracy which is higher now with 2 clients from the 10 client experiment.

05

PART 5

Simulation & Future

FL Fine-Tune System as M/G/1 Queue

Why this Queue applies here?

M (Arrivals): Clients finish local training at random times.

G (Service): Network speed over ngrok is unstable and volatile. Also VRAM and hardware in general is a big constraint in this project.

1 (Server): One central server aggregates all updates.

M/G/1 Total Delay Formulation

$$T = \frac{1}{\mu} + \frac{\lambda E[X^2]}{2(1-\rho)} < \infty \text{ if } (\rho < 1) \text{ and } (E[X^2] < \infty)$$

T: Total average delay a packet spends in the system.

1/μ : Just service time (distributed delay + VRAM training time).

E[X²] = σ² + E[X]²: Network instability from ngrok (when distributed) + Squared average time of service. It must always be a number.

ρ: Server workload utilization factor. It must always be under 1.

Understanding hardware bottlenecks

Heavy Models = Massive Workload: Checkpoint files and Fine-Tune operations become heavier.

Processing Time grows drastically: Upload and processing times (1/μ) grow drastically.

System approaches utilization limit: Server workload factor (ρ) approaches its critical limit which is 1.

Asymptotic Delay Explosion: The denominator (1-ρ) drops to zero. E[X] is inflated with bigger models and maybe bad networking connection.

The Crash: Total delay explodes to infinity causing VRAM overflows and network timeouts.

Network Perspective Conclusion: That's why the experiment took place with tiny-bert and not with a heavier model.

LoRA as the Solution

Parameter Reduction: LoRA reduces the trainable parameters by almost 99%, significantly shrinking the parameters size.

Service Time Compressing: Both network transfer time (1/μ) and server workload utilization factor (ρ) are minimized.

Delay is Limited: (1-ρ) approaches 1 and average time of the service is reduced.

Result: Total Delay (T) is limited

FL Fine-Tune System under Little's Law

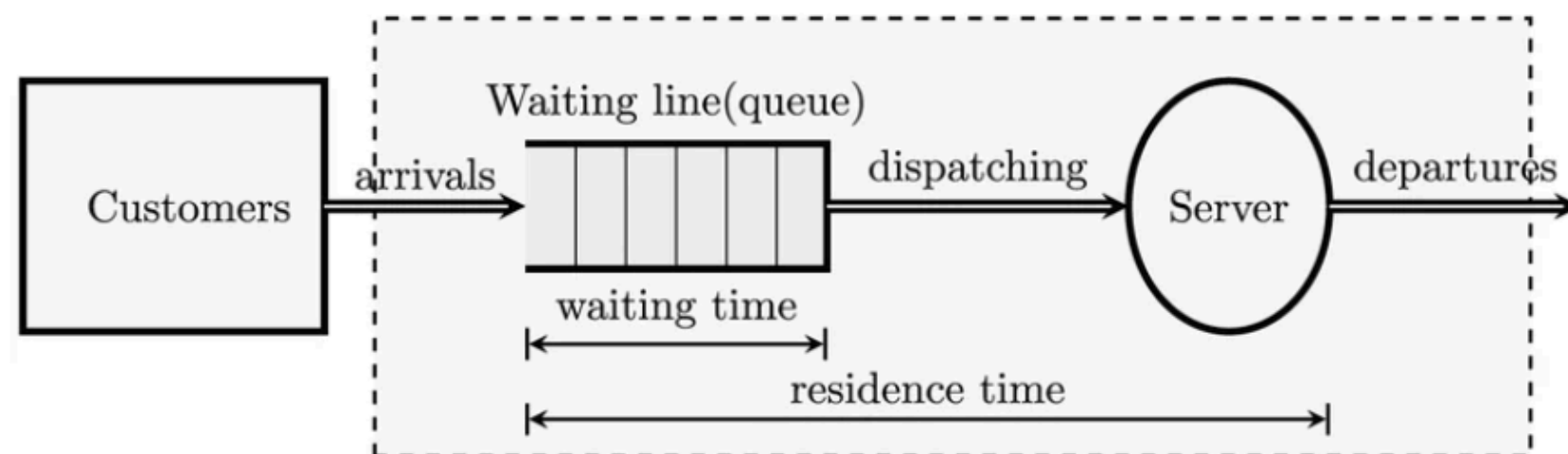
Little's Law Application

$$N = \lambda T$$

N: Average number of packets in system. Meaning total inside the queue + server. One queue packet refers to the complete set of trained weights (the checkpoint file) sent by a single client. The number of packets inside the server depends on the parallelism we defined for the hardware to operate on.

T: Average amount of time a packet spends in the system. This is measured from the moment a client (customer) requests to upload its weights, stands in the M/G/1 queue, sends the weights, up until the server finishes the FedAvg aggregation.

λ : Arrival rate of packets into the system. More specifically, in this FL System, it is the rate at which the clients complete their local training and ping the server to upload their weights. Logically, λ decreases as workload increases, but it is a stable number depending on hardware strength.



Observations via Little's Law

- The **Full Fine-Tuning** strategy makes a single packet from a client require huge workload (N increases fast).
- According to Little's Law, when the number of packets in the system increases, delay (T) at the system also explodes.
- Mathematically this explains why the system crashes when a relative heavy model is used without having the required hardware.
- This observation also concerns the ngrok channel at truly distributed training where the connection drops under big workload.

LoRA as the Solution

Parameter Reduction: **LoRA** reduces the trainable parameters by almost 99%, significantly shrinking the parameters size.

Service Time Compressing: As parameters decrease dramatically, T goes towards 0. Making the system (GPU and distributed connection) able to function.

Result: As we saw at M/G/1 queue simulation as well, **LoRA** is essential in Federated Learning Fine-Tuning. Until Quantum Networks are deployed...

Quantum Federated Learning (QFL)

What is QFL?

Quantum Federated Learning (QFL) is an emerging framework that combines decentralized Quantum Computing with Federated Learning. Edge clients train models locally, but instead of classical streaming, their trained weights are securely transmitted to the server via quantum teleportation over quantum networks.

Motivation for QFL

- **Quantum Speedups & Efficiency:** It utilizes Quantum Computing principles to process complex ML tasks much faster than classical computers, reducing local training times a lot.
- **Reduced Bandwidth Bottlenecks:** Using quantum states, we can represent complex ML information more efficiently, which will help reduce the heavy data transfer needed in classical Full Fine-Tuning.
- **Enhanced Communication Security:** FL is already known for data privacy and security. By utilizing quantum networks and quantum teleportation, the transmission of trained weights becomes more secure against any cyber threats.

Current hardware limitations and proposal

Today, achieving Quantum Communication relies on sending single photons through fiber cables, where they always suffer from signal loss and noise. To beat this distance limitation, current technology forces the chaining of multiple repeaters together using Entanglement Swapping. However, every swap multiplies the noise and damages the end-to-end network fidelity according to the formula:

$$F_s(F_1, F_2) = F_1 F_2 + (1 - F_1)(1 - F_2)$$

As we add more sequential hops, the total fidelity decays exponentially towards 0.5, turning the channel into random noise and making teleportation mathematically impossible. To scale QFL, technological improvements must take place in two stages: **Entanglement Distribution (The Split)** and **Quantum Teleportation (The Transport)**. Firstly, the generation of the Bell pairs and the Split of their qubits must avoid terrestrial fibers to minimize photon loss (for example, develop a **Satellite Quantum Network**). Secondly, the actual model Transport must minimize multi-hop repeaters to eliminate throughput-destroying purification cycles, ensuring the end-to-end fidelity remains safely above the threshold of 0.5.

The Vision of QFL Systems

Merging Quantum Networks with Federated Learning is a massive step forward in modern engineering, potentially enabling faster distributed learning. Teleporting full models over quantum networks unlocks fast, secure, and large-scale knowledge acquisition. As a result, efficiency techniques like LoRA may become less critical in this environment, since there is no longer a need to compress parameters just to save network bandwidth.

Questions & Discussion

Thank you for your attention!

Performance Analysis of Complex Networked Systems | Department of Informatics | AUEB 2026

GitHub repo i used as baseline for my project:

<https://github.com/flwrlabs/flower/tree/main/examples/flower-simulation-step-by-step-pytorch>

My GitHub repo:

<https://github.com/Michail-Marakis/Performance-Analysis-Complex-Networked-Systems>

Studied QFL from this paper:

<https://arxiv.org/pdf/2508.15998>

ChatGPT was used in order to learn the Flower framework and to experiment here by transforming my baseline code into the code that i used for the project.